

LEAKGUARD

Professional Overview & System Workflow

1. EXECUTIVE OVERVIEW

LeakGuard is a Python-first, AST-based static resource-leak detection platform for identifying unclosed resources before unsafe code reaches production. The core analyzer structurally parses Python source, detects resource acquisition, tracks resource lifecycle, analyzes realistic execution paths, classifies findings by confidence, and integrates with developer and CI/CD workflows.

Core value: **Detect early** → **explain clearly** → **block safely**.

2. PROBLEM → SOLUTION

Problem	LeakGuard Response	Usability
Resources remain open	Track resources from acquisition to cleanup.	Find lifecycle failures early.
Early returns bypass close()	Analyze control-flow paths.	Show the exact problematic path.
Exceptions bypass cleanup	Analyze try/except/finally paths.	Reduce missed leaks.
Pattern matching creates noise	Use AST + path-aware logic.	Improve precision and trust.
Need fast feedback	CLI + pre-commit integration.	Catch issues before commit.
Need enforcement	GitHub Actions + exit codes.	Block unsafe builds when configured.

3. HIGH-LEVEL WORKFLOW

Python Repository	AST Parsing	Resource Detection	Lifecycle Tracking	Control-Flow Analysis	Leak Decision	Confidence + Report	Pre-commit / CI	PASS / BLOCK
-------------------	-------------	--------------------	--------------------	-----------------------	---------------	---------------------	-----------------	--------------

The security-critical analyzer remains independent from the optional dashboard. CLI and CI/CD remain usable even without the dashboard.

4. DETAILED ANALYSIS WORKFLOW

Step	Processing	Result
1. Select Target	CLI receives a Python project directory or supported files.	Defined scan scope
2. Parse Source	Python AST creates a structural representation.	AST
3. Detect Resources	Identify open(), supported database connectors, sockets and configured custom openers.	Resource + line metadata
4. Track Lifecycle	Associate resources with acquisition and cleanup operations.	OPEN / CLOSED / uncertain
5. Analyze Paths	Evaluate sequential code, if/else, returns, try/except/finally and supported exception paths.	Path evidence
6. Verify Safe Patterns	Recognize context managers and guaranteed finally cleanup.	Reduced false positives
7. Decide Leak	Determine whether a supported path can exit with an open resource.	SAFE / LEAK / uncertain
8. Score Confidence	Assign High, Medium, Low or Safe.	Confidence level
9. Generate Report	Provide file, line, resource type, reason and remediation.	Actionable finding
10. Enforce	Apply configured threshold and exit-status policy.	PASS / FAIL

5. SYSTEM ARCHITECTURE

React Dashboard (Optional)	REST API	FastAPI Backend	LeakGuard Python Engine	AST Parser	Resource Tracker	Control-Flow Analyzer	Leak + Confidence Engine	CLI / pre-commit / GitHub Actions
----------------------------	----------	-----------------	-------------------------	------------	------------------	-----------------------	--------------------------	-----------------------------------

Architecture principle: keep the security-critical analysis engine independent from the optional dashboard. This preserves direct developer and CI/CD usability.

6. TECHNOLOGY & INTEGRATIONS

Layer	Technology	Purpose / Integration
Frontend	React + Vite + Tailwind CSS	Interactive scan results, project overview and analytics.
Backend	FastAPI + Pydantic	REST API between dashboard and analysis engine.
Core Engine	Python + Python AST	Structural source analysis and resource tracking.
Path Analysis	Custom Python logic	Branches, early returns and exception-path reasoning.
CLI	Typer / Click + Rich	Installable scanner with readable local/CI output.
Testing	Pytest	Unit and integration testing.
CI/CD	GitHub Actions	Automatic repository scanning and build blocking.
Local Gate	pre-commit	Catch issues before code is committed.
Database	Supabase / PostgreSQL (Optional)	Scan history, projects, reports and trends.
Frontend Tooling	Node.js + npm	React dependency, development and build tooling.

7. DEVELOPER + CI/CD WORKFLOW

Write Code	Local Scan	Review Findings	Tests	pre-commit	Push / PR	GitHub Actions	Threshold Check	PASS / BLOCK
------------	------------	-----------------	-------	------------	-----------	----------------	-----------------	--------------

Recommended hackathon configuration: block high-confidence findings. Medium and low-confidence findings can be reported or configured by policy. Blocking results must return a non-zero exit status.

8. REPORTING & USER EXPERIENCE

Finding field	Purpose
File + line	Immediately locate the opening expression.
Resource type	Identify file, database connection, socket or configured resource.
Tracked variable	Connect acquisition and cleanup evidence.
Execution-path reason	Explain why cleanup may not be reached.
Confidence	Communicate strength of the analysis.
Suggested remediation	Provide a practical correction where possible.

Example: **LEAKGUARD ANALYSIS FAILED** — database.py — Database Connection — Opened: Line 42 — Early return can exit before connection.close() — Confidence: HIGH — Suggested action: close before returning or use a context-managed/finally-based cleanup pattern.

9. SAFE PATTERNS

Context Manager (with)	Try / finally	Explicit close()	Path Verification	SAFE when cleanup is guaranteed
------------------------	---------------	------------------	-------------------	---------------------------------

Explicit close() is safe only when the supported analysis model proves all relevant paths reach cleanup.

10. CLASSIFICATION MODEL

Level	Meaning	Typical action
High / Definite	Confirmed execution path exits without cleanup.	Block CI when configured.
Medium / Likely	Realistic branch or exception path may bypass cleanup.	Warn or configure threshold.
Low / Possible	Resource ownership becomes uncertain.	Informational / review.
Safe	Cleanup is structurally guaranteed.	No leak report.

11. TESTING & QUALITY WORKFLOW

Seed Leaks	Seed Safe Cases	Run Pytest	Collect Findings	Compare Expected	Measure TP / FP / FN	Document Limits	Regression Suite
------------	-----------------	------------	------------------	------------------	----------------------	-----------------	------------------

Required scenarios include missing close(), early return, branch bypass, exception path, database connection, socket, safe context-manager usage, safe try/finally, explicit cleanup and uncertain cross-function ownership.

12. KNOWN LIMITATIONS

Limitation	Required handling
Cross-function ownership	May not be fully tracked in MVP; use lower confidence or document scope.
Object attributes / collections	May require deeper object-state analysis.
Dynamic Python behavior	Can reduce static-analysis certainty.
Third-party resource APIs	May require explicit configuration.
Interprocedural analysis	MVP primarily focuses on intra-procedural analysis.

13. HACKATHON DEMONSTRATION

Leaky Code	LeakGuard Scan	Exact Finding	Fix Safe Pattern	Re-scan	SAFE	Push PR	CI PASS
------------	----------------	---------------	------------------	---------	------	---------	---------

Demonstrate an obvious leak, safe context manager, early-return leak, exception-path case, corrected code, failing GitHub Actions run and passing run after the fix.

14. MVP PRIORITY

Priority	Component	Reason
1	Python AST Analyzer	Core security product.
2	Resource + Path Tracking	Required for real leak detection.
3	CLI + Reports	Makes the tool usable and demonstrable.
4	GitHub Actions	Demonstrates real CI build blocking.
5	pre-commit	Improves local developer workflow.
6	Confidence Scoring	Reduces alert fatigue.
7	React Dashboard	Improves visualization and presentation.
8	Database	Useful for history, not essential for core MVP.

15. FINAL PRODUCT DEFINITION

LeakGuard = Python AST Analysis + Resource Tracking + Control-Flow Analysis + Actionable CLI + Confidence Scoring + Pre-commit/GitHub Actions Build Blocking + Seeded Test Repository.

Python performs the security analysis, FastAPI exposes the engine, React provides optional visualization, and GitHub Actions / pre-commit integrate LeakGuard into the software development lifecycle.